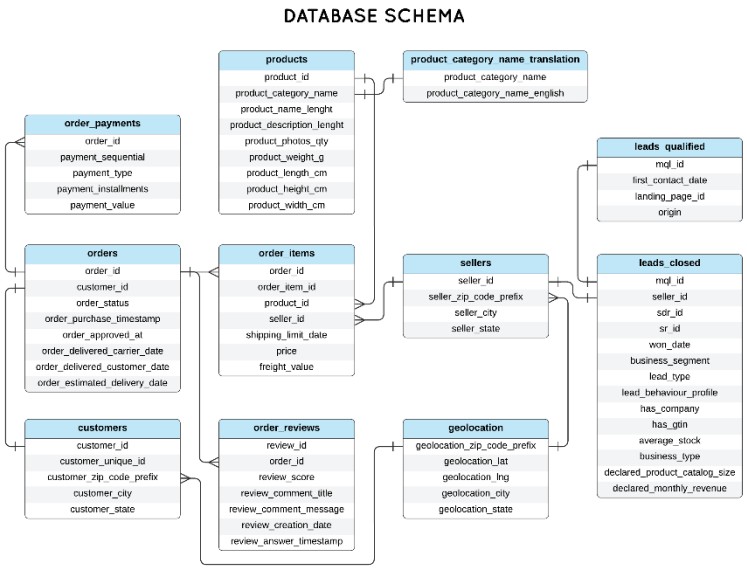
Data Engineering Project Documentation

- **Database Schema (Source)**



- **Data Warehouse Schema (Target)**

I went with Star Schema to simplify query logic and enhance performance by minimizing joins. This single-fact design avoids over-engineering while providing a direct path for business users to answer key questions on sales and delivery. It ensures the solution is intuitive, scalable, and easy to maintain.

1. Architecture Diagram

The system follows a modular ETL (Extract-Transform-Load) architecture designed for a clear separation of concerns.

- **Source Layer**: A static SQLite database containing normalized tables representing e-commerce operations.
- **Orchestration Layer**: A Python-based engine using the schedule library to trigger the ETL process every 60 seconds.
- **Transformation Layer**: A modular Python pipeline utilizing Pandas for data cleaning and metric calculation.
- **Warehouse Layer**: A PostgreSQL instance hosting a Star Schema optimized for analytical reporting.

---

2. Data Model (Star Schema)

I transitioned the data from a normalized source into a **Kimball Star Schema**. This simplifies the data for business users and improves query performance by reducing the number of complex joins.

Fact Table (fact_sales_details):

- **Grain**: One row per order item.
- **Metrics**: Includes item_price, freight_value, total_item_revenue, and delay_days.
- **Keys**: Foreign keys linking to all dimensions and a unique surrogate key.

Dimension Tables:

- **dim_customer**: Geography details of buyers.
- **dim_product**: Product categories translated to English.
- **dim_seller**: Seller location details.
- **dim_date**: A generated calendar table for time-series analysis.

---

3. Pipeline Design

The pipeline is designed as a **Full Load (Truncate & Insert)** batch process.

1. **Extraction**: Pulls raw data from SQLite using SQLAlchemy.
2. **Transformation**:
    - I handled data quality by mapping category translations to English.
    - I calculated delay_days by comparing actual vs. estimated delivery timestamps to provide performance insights.
    - I mapped source business keys to Data Warehouse surrogate keys.
3. **Loading**:
    - I used TRUNCATE ... RESTART IDENTITY CASCADE to clear the warehouse while maintaining schema integrity.
    - I implemented **Chunking** to ensure the pipeline can handle the dataset efficiently without memory overflow.

---

4. Key Assumptions

- **Static Historical Data**: I assumed that once an order is "Delivered," its historical attributes do not change, allowing for a Full Load approach.
- **Timezone Consistency**: I assumed timestamps are in a consistent format as provided by the Olist source.
- **Core Process**: I focused on the "Order Fulfillment" process as the primary business activity.

---

5. Trade-offs

- **Full Load vs. Incremental**: I chose a **Full Load** strategy. While incremental loads save bandwidth, a full load is easier to maintain and ensures 100% data consistency for this dataset size.
- **Kimball vs. Inmon**: I selected the **Kimball (Star Schema)** approach. This is better suited for rapid analytical reporting, whereas the Inmon approach would be over-engineered for the current requirements.
- **Python vs. Heavy Orchestrators**: I utilized a custom Python orchestrator with the schedule library. This provides a lightweight, easy-to-deploy solution without the infrastructure overhead of tools like Airflow.